

Programming Foundations, Compiler & Data Structures

Lesson 51 — Languages, Translators and Basic Structures

JKSSB Computer Awareness

October 8, 2026

Why Programming Languages Have Levels

- ▶ A computer finally understands only **machine language**: sequences of binary 0s and 1s.
- ▶ Writing directly in 0s and 1s is slow, error-prone, and machine-specific.
- ▶ Programs are therefore written in easier languages and then **translated** into machine language.
- ▶ Languages are grouped by **level**: low-level (machine, assembly) and high-level.
- ▶ Lower level = closer to hardware; higher level = closer to human language.

The one idea

Every program must finally become machine language before the CPU can run it.

Machine Language

- ▶ **Machine language** is the only language the CPU executes directly.
- ▶ It is written in **binary**: sequences of 0s and 1s.
- ▶ It is **machine-dependent**: each processor family has its own instruction set.
- ▶ It is fast (no translation needed) but very difficult to write and debug.
- ▶ Also called the **first-generation language**, and it is a low-level language.

Assembly Language

- ▶ **Assembly language** uses short **mnemonics** such as ADD, MOV, and SUB instead of binary codes.
- ▶ It is still **machine-dependent**: one assembly language per processor family.
- ▶ It maps closely to machine instructions: about one mnemonic per machine instruction.
- ▶ It must be translated into machine language by an **assembler**.

Level check

Assembly language is a **second-generation** (low-level) language, not a high-level language.

High-Level Languages

- ▶ **High-level languages** use English-like words and mathematical notation.
- ▶ Examples: **C, C++, Java, Python, COBOL, FORTRAN**.
- ▶ They are **machine-independent**: the same program runs on different machines after translation.
- ▶ They are easier to learn, write, read, and maintain.
- ▶ They must be translated by a **compiler** or an **interpreter**.

Translators: Assembler, Compiler, Interpreter

- ▶ A **translator** converts a program from one language to another, usually into machine language.
- ▶ **Assembler**: translates **assembly language** into machine code.
- ▶ **Compiler**: translates a **high-level** program into machine code **as a whole**, producing object code.
- ▶ **Interpreter**: translates and executes a high-level program **line by line**.
- ▶ All three produce a runnable result; they differ in input language and method.

Compiler vs Interpreter

Point	Compiler	Interpreter
Input	Whole program at once	One statement at a time
Output	Object code file	No separate object file
Speed	Faster to execute	Slower to execute
Errors	Reported after full compile	Reported line by line
Debugging	Harder	Easier

Trap alert

A compiler translates the **whole program at once**; an interpreter works **line by line**.

Source Code, Object Code, Machine Code

- ▶ **Source code**: the program as written by the programmer in a high-level or assembly language; it is human-readable.
- ▶ The compiler translates source code into **object code**.
- ▶ **Object code**: machine-readable, relocatable code produced by the translator; it is not yet linked.
- ▶ **Machine code**: the final binary instructions the CPU executes directly.
- ▶ Source code is human-readable; object and machine code are machine-readable.

Algorithm

- ▶ An **algorithm** is a **finite, step-by-step** procedure to solve a problem.
- ▶ It has a clear **start** and **end**, and every step is unambiguous.
- ▶ It is language-independent: it can be written in plain English or pseudocode.
- ▶ A program is simply the algorithm expressed in a programming language.

Worked example

Add two numbers: (1) start, (2) read a and b , (3) compute $sum = a + b$, (4) display sum , (5) stop.

Flowchart

- ▶ A **flowchart** is a **pictorial** representation of an algorithm.
- ▶ It uses standard symbols joined by arrows to show the flow of control.

Symbol	Meaning
Oval (terminator)	Start / End
Parallelogram	Input / Output
Rectangle	Process (computation)
Diamond (rhombus)	Decision
Arrow	Flow line

Debugging and Types of Errors

- ▶ **Debugging** is the process of finding and fixing **errors (bugs)** in a program.
- ▶ **Syntax error**: a grammar or spelling mistake; the compiler catches it (e.g. a missing semicolon).
- ▶ **Logical error**: the program runs but gives a wrong result; the hardest to find.
- ▶ **Runtime error**: occurs while the program is running (e.g. division by zero).

Trap alert

A logical error is **not** reported by the compiler — the program compiles and runs but the answer is wrong.

Object-Oriented Programming (OOP) Basics

- ▶ **OOP** stands for **Object-Oriented Programming**.
- ▶ **Class**: the blueprint or template; **Object**: an instance of a class.
- ▶ **Encapsulation**: binding data and methods together, with data hiding.
- ▶ **Inheritance**: a class acquires the properties of another class.
- ▶ **Polymorphism**: one name, many forms.
- ▶ **Abstraction**: hiding implementation details and showing only the essential features.

Keep the pair straight

A class is the blueprint; an object is the thing built from it.

Stack and LIFO

- ▶ A **stack** is a linear data structure that works on **LIFO**: Last In, First Out.
- ▶ Insertion is called **PUSH**; deletion is called **POP**.
- ▶ Operations take place at one end, the **top**; the other end is the bottom.
- ▶ The last element pushed is the first one popped.

Office example

A stack of files: the last file placed on top is the first one taken off.

Basic Data Structures

Structure	Defining property
Array	Fixed-size, same-type elements in contiguous locations; accessed by an index
Stack	LIFO; push and pop at the top
Queue	FIFO; insertion at the rear, deletion at the front
Linked list	Nodes holding data and a pointer; not contiguous; can grow dynamically
Tree	Hierarchical; a root with child nodes

Data-Structure Identification

- ▶ To identify a structure, look for its **defining rule**:
 - ▶ same type + fixed size + index → **array**.
 - ▶ Last In First Out → **stack**.
 - ▶ First In First Out (a queue at a counter) → **queue**.
 - ▶ data plus pointer nodes, dynamic → **linked list**.
 - ▶ root and branches, hierarchy → **tree**.

Trap alert

A stack is LIFO; a queue is FIFO. Do not swap them.

Trap Alert: Terms to Keep Straight

- ▶ Compiler: whole program at once; interpreter: line by line.
- ▶ Assembler translates assembly; compiler and interpreter translate high-level code.
- ▶ Source code is human-readable; object and machine code are machine-readable.
- ▶ Algorithm is a step-by-step procedure; a flowchart is its diagram.
- ▶ Stack is LIFO; queue is FIFO.
- ▶ A logical error is not caught by the compiler.

Lesson 51: Recall

- ▶ Languages by level: machine (1st, binary), assembly (2nd, mnemonics), high-level (3rd).
- ▶ Translators: assembler (assembly), compiler (whole high-level program), interpreter (line by line).
- ▶ Source code is human-readable; object and machine code are machine-readable.
- ▶ Algorithm = step-by-step procedure; flowchart = diagram with standard symbols.
- ▶ Debugging removes syntax, logical, and runtime errors.
- ▶ OOP: class/object, encapsulation, inheritance, polymorphism, abstraction.
- ▶ Stack = LIFO (push/pop); queue = FIFO; arrays, linked lists, and trees.